

Kacper Mierzwa
Informatyka Techniczna
III rok, gr. 6
09.01.2025

Metoda Elementów Skończonych

SYMULACJA USTALONYCH PROCESÓW CIEPLNYCH

1. Cel sprawozdania

Celem sprawozdania jest przedstawienie wyników oraz analizy symulacji ustalonych procesów cieplnych przeprowadzonych z wykorzystaniem Metody Elementów Skończonych (MES). W ramach pracy nacisk zostanie położony na praktyczne aspekty obliczeń i symulacji, ze szczególnym uwzględnieniem analizy wpływu wybranych parametrów modelowania i metod numerycznych na dokładność oraz stabilność uzyskanych wyników. Sprawozdanie ma służyć zarówno jako dokumentacja wyników symulacji, jak i jako narzędzie oceny efektywności stosowanych metod numerycznych w kontekście problemów cieplnych. Główne cele sprawozdania wymieniono poniżej.

Praktyczna implementacja MES dla procesów cieplnych

Wykorzystanie MES do rozwiązania równań przewodzenia ciepła w stanie ustalonym, w tym budowę macierzy przewodzenia ciepła $[H]$, macierzy pojemności cieplnej $[C]$, oraz wektorów obciążeń cieplnych $\{P\}$, które determinują zachowanie modelowanego systemu. Implementacja procedur numerycznych związanych z całkowaniem funkcji kształtu oraz ich pochodnych w przestrzeni lokalnej i globalnej.

Analiza wpływu siatki elementów skończonych na wyniki symulacji

Badanie zależności wyników od gęstości siatki (np. siatki o podziałach 4x4, 4x4_mix i 31x31), co pozwoli ocenić wpływ wielkości elementów na dokładność obliczeń i zbieżność wyników.

Ocena schematów całkowania

Porównanie wyników uzyskanych dla różnych schematów całkowania numerycznego, takich jak 2-punktowy, 3-punktowy oraz 4-punktowy schemat Gaussa, co pozwala określić optymalny kompromis między dokładnością a złożonością obliczeniową.

Uwzględnienie warunków brzegowych

Implementacja zewnętrznych warunków brzegowych, takich jak wymiana ciepła przez konwekcję i promieniowanie, w celu dokładniejszego odwzorowania rzeczywistych warunków pracy badanego systemu.

Ocena wpływu parametrów termicznych materiału

Analiza wpływu parametrów, takich jak współczynnik przewodzenia ciepła k , pojemność cieplna c_p oraz gęstość ρ na rozkład temperatury w modelu.

Rozwiązanie układów równań wynikających z agregacji macierzy

Przeprowadzenie obliczeń w układzie globalnym dla agregowanych macierzy $[H]$, $[C]$ i wektorów $\{P\}$, uwzględniając iteracyjne rozwiązania równań węzłowych.

Porównanie wyników symulacji z testami referencyjnymi

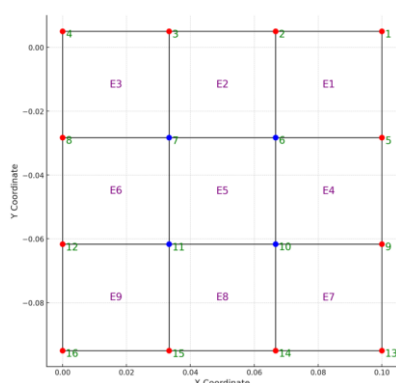
Weryfikacja wyników symulacji za pomocą porównania ich z wynikami testów referencyjnych (z wynikami ze strony UPEL przedmiotu) w celu oceny poprawności i wiarygodności modelu.

Opracowanie praktycznych wniosków

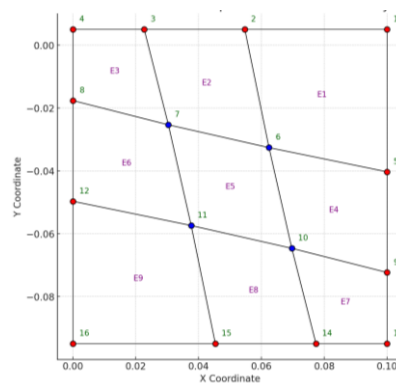
Identyfikacja potencjalnych źródeł błędów oraz ograniczeń w zastosowanej metodologii. Propozycje usprawnień dla algorytmów i procedur MES w kontekście symulacji procesów cieplnych.

2. Wstęp

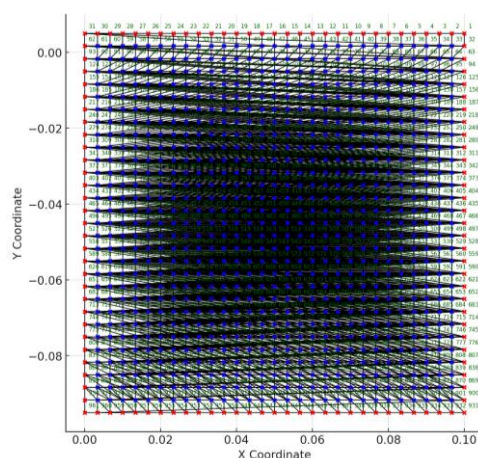
Rozpatrywany obszar geometryczny Ω dzielony jest na n elementów skończonych. W tym sprawozdaniu będą brane pod uwagę trzy takie siatki:



Rysunek 1: Siatka Test1_4_4.txt



Rysunek 2: Siatka Test2_4_4_MixGrid.txt



Rysunek 3: Siatka Test3_31_31_kwadrat.txt

W każdym elemencie wprowadza się funkcje kształtu N_i , które interpolują wartość analizowanej wielkości w węzłach elementu. Dodatkowo w zależności od liczby punktów całkowania podstawiane są różne wartości ξ i η . Przedstawia to tabela:

N	k	Węzły x_k	Współczynniki A_k
1	0; 1	$\pm \frac{1}{\sqrt{3}}$	1
2	0; 2	$\pm \frac{\sqrt{3}}{5}$	$\frac{5}{9}$
	1	0	$\frac{8}{9}$
3	0; 3	$\pm \sqrt{\left(\frac{3}{7} - \frac{2}{7}\sqrt{\frac{6}{5}}\right)}$	$\frac{18 + \sqrt{30}}{36}$
	1; 2	$\pm \sqrt{\left(\frac{3}{7} + \frac{2}{7}\sqrt{\frac{6}{5}}\right)}$	$\frac{18 - \sqrt{30}}{36}$
4	0; 4	0	$\frac{128}{225}$
	1; 3	$\pm \frac{1}{3} \sqrt{\left(5 - 2\sqrt{\frac{10}{7}}\right)}$	$\frac{322 + 13\sqrt{70}}{900}$
	2	$\pm \frac{1}{3} \sqrt{\left(5 + 2\sqrt{\frac{10}{7}}\right)}$	$\frac{322 - 13\sqrt{70}}{900}$

Tabela 1: Węzły i współczynniki kwadratur Gaussa-Legendre'a

Dla elementu czterowęzłowego funkcje kształtu są definiowane jako:

$$N_1 = \frac{1}{4}(1 - \xi)(1 - \eta) \quad (1.1)$$

$$N_2 = \frac{1}{4}(1 + \xi)(1 - \eta) \quad (1.2)$$

$$N_3 = \frac{1}{4}(1 + \xi)(1 + \eta) \quad (1.3)$$

$$N_4 = \frac{1}{4}(1 - \xi)(1 + \eta) \quad (1.4)$$

gdzie ξ i η to współrzędne lokalne w układzie elementu.

Proces przewodzenia ciepła w stanie ustalonym opisuje równanie Fouriera:

$$\text{div}(k(y)\text{grad}(t)) + Q = 0 \quad (2.1)$$

które może być zapisane też w postaci:

$$\frac{\partial}{\partial x} \left(k_x(t) \frac{\partial t}{\partial x} \right) + \frac{\partial}{\partial y} \left(k_y(t) \frac{\partial t}{\partial y} \right) + \frac{\partial}{\partial z} \left(k_z(t) \frac{\partial t}{\partial z} \right) + Q = 0 \quad (2.2)$$

gdzie: t – temperatura w obszarze; k – współczynnik przewodzenia ciepła; Q – prędkość generacji ciepła

Przykładowe dane testowe

SimulationTime 500	Czas symulacji - τ [s]
SimulationStepTime 50	Krok symulacji - $\Delta\tau$ [s]
Conductivity 25	Przewodność cieplna - $k(t)$ [W/(m° C)]
Alfa 300	Współczynnik wymiany ciepła - α [W/m²K]
Tot 1200	Temperatura otoczenia - t_{ot} [C]
InitialTemp 100	Temperatura początkowa - t_0 [C]
Density 7800	Gęstość - ρ [kg/m³]
SpecificHeat 700	Ciepło właściwe - c_p [J/(kg° C)
Nodes number 16	Liczba węzłów
Elements number 9	Liczba elementów
*Node	Lista współrzędnych węzłów
1, 0.100000001, 0.00499999989	
2, 0.0666666701, 0.00499999989	
3, 0.0333333351, 0.00499999989	
4, 0., 0.00499999989	
5, 0.100000001, -0.0283333343	
6, 0.0666666701, -0.0283333343	
7, 0.0333333351, -0.0283333343	
8, 0., -0.0283333343	
9, 0.100000001, -0.0616666675	
10, 0.0666666701, -0.0616666675	
11, 0.0333333351, -0.0616666675	
12, 0., -0.0616666675	
13, 0.100000001, -0.0949999988	
14, 0.0666666701, -0.0949999988	
15, 0.0333333351, -0.0949999988	
16, 0., -0.0949999988	
*Element, type=DC2D4	Lista elementów w siatce
1, 1, 2, 6, 5	
2, 2, 3, 7, 6	
3, 3, 4, 8, 7	
4, 5, 6, 10, 9	
5, 6, 7, 11, 10	
6, 7, 8, 12, 11	
7, 9, 10, 14, 13	
8, 10, 11, 15, 14	
9, 11, 12, 16, 15	
*BC	
1, 2, 3, 4, 5, 8, 9, 12, 13, 14, 15, 16	Lista węzłów brzegowych

Do obliczenia pochodnych względem ξ oraz η korzystamy z zależności:

$$\frac{\partial N(x,y)}{\partial \xi} = \frac{\partial N(x,y)}{\partial x} \frac{\partial x}{\partial \xi} + \frac{\partial N(x,y)}{\partial y} \frac{\partial y}{\partial \xi} \quad (3.1)$$

$$\frac{\partial N(x,y)}{\partial \eta} = \frac{\partial N(x,y)}{\partial x} \frac{\partial x}{\partial \eta} + \frac{\partial N(x,y)}{\partial y} \frac{\partial y}{\partial \eta} \quad (3.2)$$

Ostatecznie obliczamy pochodne funkcji kształtu używając wzorów:

$$\frac{\partial N_1}{\partial \xi} = -\frac{1}{4}(1-\eta) \quad \frac{\partial N_2}{\partial \xi} = \frac{1}{4}(1-\eta) \quad \frac{\partial N_3}{\partial \xi} = \frac{1}{4}(1+\eta) \quad \frac{\partial N_4}{\partial \xi} = -\frac{1}{4}(1+\eta) \quad (4.1)$$

$$\frac{\partial N_1}{\partial \eta} = -\frac{1}{4}(1-\xi) \quad \frac{\partial N_2}{\partial \eta} = -\frac{1}{4}(1+\xi) \quad \frac{\partial N_3}{\partial \eta} = \frac{1}{4}(1+\xi) \quad \frac{\partial N_4}{\partial \eta} = \frac{1}{4}(1-\xi) \quad (4.2)$$

Po czym obliczamy jacobian przekształcenia dla każdego punktu:

$$\frac{\partial x}{\partial \xi} = \frac{\partial N_1}{\partial \xi} x_1 + \frac{\partial N_2}{\partial \xi} x_2 + \frac{\partial N_3}{\partial \xi} x_3 + \frac{\partial N_4}{\partial \xi} x_4 \quad (5.1)$$

$$\frac{\partial x}{\partial \eta} = \frac{\partial N_1}{\partial \eta} x_1 + \frac{\partial N_2}{\partial \eta} x_2 + \frac{\partial N_3}{\partial \eta} x_3 + \frac{\partial N_4}{\partial \eta} x_4 \quad (5.2)$$

$$\frac{\partial y}{\partial \xi} = \frac{\partial N_1}{\partial \xi} y_1 + \frac{\partial N_2}{\partial \xi} y_2 + \frac{\partial N_3}{\partial \xi} y_3 + \frac{\partial N_4}{\partial \xi} y_4 \quad (5.3)$$

$$\frac{\partial y}{\partial \eta} = \frac{\partial N_1}{\partial \eta} y_1 + \frac{\partial N_2}{\partial \eta} y_2 + \frac{\partial N_3}{\partial \eta} y_3 + \frac{\partial N_4}{\partial \eta} y_4 \quad (5.4)$$

Mając wszystkie dane należy też obliczyć wyznacznik jacobianu. Następnie wykorzystując wcześniej obliczone pochodne funkcji kształtu, można wyznaczyć macierz $[H]$ dla każdego punktu całkowania w elemencie(6). Obliczenie macierzy $[H]$ dla każdego elementu to suma macierzy wszystkich punktów.

$$[H] = \int_V^k(t) \left(\left\{ \frac{\partial \{N\}}{\partial x} \right\} \left\{ \frac{\partial \{N\}}{\partial x} \right\}^T + \left\{ \frac{\partial \{N\}}{\partial y} \right\} \left\{ \frac{\partial \{N\}}{\partial y} \right\}^T \right) dV \quad (6)$$

Kolejno należy obliczyć macierz $[H_{bc}]$ dla każdego elementu. Aby to wykonać należy obliczyć takie macierze dla wszystkich punktów całkowania (7) w każdym brzegowego boku elementu z wykorzystaniem funkcji kształtu, a następnie je zsumować. W podobny sposób obliczany jest wektor $\{P\}$ (8) oraz macierz $[C]$ (9).

$$[H_{bc}] = \int_S \alpha(\{N\}\{N\}^T) ds = \sum_{i=1}^{n_{pc}} f(pc_i) w_i \det[J] \quad (7)$$

$$[P] = \int_S \alpha\{N\} t_{ot} ds = \sum_{i=1}^{n_{pc}} f(pc_i) w_i \det[J] \quad (8)$$

$$[C] = \int_V \rho c_p (\{N\} \{N\}^T) dV \quad (9)$$

Obliczoną macierz $[H_{bc}]$ reprezentującą wpływ konwekcji należy ją dodać do macierzy $[H]$ (9) i stworzyć globalną macierz $[H]$ (macierz sztywności dla całego obszaru) oraz wektor $\{P\}$ (wektor obciążeń strumieni cieplnych dla całego układu), które zostaną uzupełnione poprzez agregację.

$$[H] = [H] + \int_S \alpha \{N\} \{N\}^T dS \quad (10)$$

Dla całej domeny Ω agreguje się macierze i wektory dla wszystkich elementów, uzyskując globalny układ równań:

$$[H]\{t\} + \{P\} = 0 \quad (11)$$

Rozwiązanie tego układu daje wartości temperatury w węzłach siatki. Finalnym rozwiązaniem jest wzór reprezentujący symulację zmiany temperatur w każdym elemencie w czasie (12).

$$\left([H] + \frac{[C]}{\Delta\tau}\right)\{t_1\} - \left(\frac{[C]}{\Delta\tau}\right)\{t_0\} + \{P\} = 0 \quad (12)$$

3. Charakterystyka oprogramowania

3.1 Informacje ogólne

```
#define npc 4

double dn1ksi(double a);
double dn2ksi(double a);
double dn3ksi(double a);
double dn4ksi(double a);
double dn1eta(double a);
double dn2eta(double a);
double dn3eta(double a);
double dn4eta(double a);

double n1(double ksi, double eta);
double n2(double ksi, double eta);
double n3(double ksi, double eta);
double n4(double ksi, double eta);
```

Fragment kodu 1: funkcje kształtu

Na początku programu ustalana jest liczba punktów całkowania ($npc = 4$ lub 9 lub 16). Zaimplementowane są również funkcje kształtu do obliczenia jacobianów oraz macierzy $[H_{bc}]$ i wektora $\{P\}$.

```

GlobalData data;
ElemUniv eu;
GaussIntegration gi;
gi.initialize();
eu.initialize(&gi);
Grid grid;

grid.obliczJakobiany(&eu);
grid.obliczNEH(&eu, data.conductivity);
grid.obliczHbcPrzezGrid(grid.node, &eu, data.alfa, data.tot, &gi);
grid.obliczC(data.specificHeat, data.density, &gi);
HGlobal hg(&data);
hg.symulacja(data.simulationTime, data.simulationStepTime, &grid, data.initialTemp);

```

Fragment kodu 2: fragment funkcji main()

Funkcja *main()* wczytuje wszystkie dane z pliku tekstowego oraz tworzy i inicjalizuje wszystkie obiekty i wywołuje finalną funkcję *symulacja()*;

3.2 Struktura GaussIntegration

```

struct GaussIntegration {
    double* w = new double[sqrt(npc)];
    double* pc = new double[sqrt(npc) + 2];

    void initialize() {}
    ~GaussIntegration() {}
};

```

Fragment kodu 3: struktura GaussIntegration

Struktura ta przechowuje wszystkie węzły x_k oraz współczynniki A_k w zależności od liczby punktów całkowania, zawiera funkcję ją inicjalizującą oraz destruktor.

3.3 Struktura ElemUniv

```

struct ElemUniv {
    double** ksi;
    double** eta;

    struct Surface {
        double** N;

        ~Surface() {}
        void initialize() {}
        void oblicz(int i, GaussIntegration* gi) {}
        void obliczHbc(double* nodex, double* nodey, bool* node2, int* pom, double** tab, int c, int z,
double* p, int t, GaussIntegration* gi) {}
    };
    Surface surface[npc];

    ~ElemUniv() {}
    void initialize(GaussIntegration* gi) {}
}

```



```
void obliczHbcDlaJednegoBoku(double* nodex, double* nodey, bool* node2, int* tab, double**
H_bc_temp, int c, double* p, int t, GaussIntegration* gi) {}
};
```

Fragment kodu 4: struktura ElemUniv

Struktura *ElemUniv* zawiera tabele pomocnicze: pochodne funkcji kształtu względem ξ oraz η , strukturę *Surface*, która jest wykorzystywana do obliczania macierzy $[Hbc]$ oraz wektora $\{P\}$, tablicę obiektów *Surface* potrzebną do obliczenia tej macierzy i wektorów dla każdego elementu. Obie struktury posiadają funkcje inicjalizujące ich obiekty oraz destruktory.

3.4 Struktura Node

```
struct Node {
    double x = 0.0;
    double y = 0.0;
    bool bc = false;
};
```

Fragment kodu 5: struktura Node

Ta struktura służy jedynie do przechowywania współrzędnych węzłów oraz informacji czy są one brzegowe.

3.5 Struktura Jakobian

```
struct Jakobian {
    double j[2][2] = { 0.0 };
    double j1[2][2] = { 0.0 };
    double detJ;

    void obliczJ(ElemUniv* eu, Node* nodes, int* tab, int pointIndex) {}
    void obliczDetJ() {}
    void obliczJ1() {}
};
```

Fragment kodu 6: struktura Jakobian

Ta struktura zawiera macierz jacobianu oraz jej wyznacznik oraz funkcje je obliczające.

3.6 Struktura Element

```
struct Element {
    int* values = new int[4];
    Jakobian* jacobian = new Jakobian[np];
    double** H = new double* [4];
    double** Hbc = new double* [4];
    double** C = new double* [4];
    double* p = new double[4] {0.0};
};
```

```

void initialize() {}
~Element() {}
void jakobiany(ElemUniv* eu, Node* nodes) {}
void obliczH(ElemUniv* eu, int conductivity) {}
void obliczHbcDlaKazdegoElementu(double* nodex, double* nodey, bool* node2, ElemUniv* euu, int c,
int t, GaussIntegration* gi) {}
void obliczCDlaKazdegoElementu(double sh, double density, GaussIntegration* gi) {}
};

```

Fragment kodu 7: struktura Element

Struktura *Element* zawiera tablicę z id tworzących ją węzłów, tablicę jacobianów dla każdego punktu całkowania, macierze $[H]$, $[Hbc]$, $[C]$, wektor $\{P\}$, funkcję inicjalizującą obiekt oraz destruktor. Dodatkowo zawiera pomocnicze metody do obliczania jacobianów oraz macierzy $[Hbc]$ wraz z wektorem $\{P\}$, a także funkcje obliczające macierz $[H]$ oraz $[C]$.

3.7 Struktura Grid

```

struct Grid {
    int nN;
    int nE;
    Element* element;
    Node* node;

    ~Grid() {}
    void initialize() {}
    void obliczJakobiany(ElemUniv* eu) {}
    void obliczNEH(ElemUniv* eu, int c) {}
    void obliczHbcPrzezGrid(Node* node, ElemUniv* euu, int c, int t, GaussIntegration* gi){}
};

```

Fragment kodu 8: struktura Grid

Ta struktura jest reprezentacją siatki - zawiera kolejno liczbę węzłów w siatce, liczbę elementów, tablicę elementów, tablicę węzłów, funkcję ją inicjalizującą oraz destruktor, a także funkcje pomocnicze do obliczania jacobianów, macierzy $[H]$ oraz macierzy $[Hbc]$.

3.8 Struktura GlobalData

```

struct GlobalData {
    int simulationTime;
    int simulationStepTime;
    int conductivity;
    int alfa;
    int tot;
    int initialTemp;
    int density;
    int specificHeat;
    int nN;
    int nE;
};

```

Fragment kodu 9: struktura GlobalData

Struktura ta zawiera większość danych, pobranych z pliku tekstowego, potrzebnych do wykonania symulacji.

3.9 Struktura HGlobal

```
struct HGlobal {
    double** values;
    double** Cg;
    int size;
    double* Pg;
    double* t;

    HGlobal(GlobalData* d) {
        size = d->nN;
        values = new double* [size];
        Cg = new double* [size];
        for (int i = 0; i < size; i++) {
            values[i] = new double[size] {0};
            Cg[i] = new double[size] {0};
        }
        Pg = new double[size] {0};
        t = new double[size] {0};
    }

    ~HGlobal() {
        for (int i = 0; i < size; i++) {
            delete[] values[i];
            delete[] Cg[i];
        }
        delete[] values;
        delete[] Cg;
        delete[] Pg;
        delete[] t;
    }

    void symulacja(int time, int step, Grid* grid, int startTemp) {
        agreguj(grid);
        double* tzero = new double[size];
        double** pom = new double* [size];
        for (int i = 0; i < size; i++) {
            tzero[i] = (double)startTemp;
            pom[i] = new double[size] {0.0};
        }
        cout << endl;
        for (int iteracja = step; iteracja <= time; iteracja += step) {
            for (int i = 0; i < size; i++) {
                for (int j = 0; j < size; j++) {
                    values[i][j] += Cg[i][j] / iteracja;
                    if (Cg[i][j] != 0.0) {
                        Pg[i] += (Cg[i][j] / iteracja) * tzero[j];
                    }
                }
            }
            obliczT(grid);
        }
    }
}
```

```

    findMinMax(t);
    for (int i = 0; i < size; i++) {
        tzero[i] = t[i];
        for (int j = 0; j < size; j++) {
            values[i][j] = 0.0;
        }
        Pg[i] = 0.0;
    }
    agreguj(grid);
}

cout << "\n\n\n";

for (int i = 0; i < size; i++) {
    delete[] pom[i];
}
delete[] pom;
delete[] tzero;
}

void findMinMax(double* tab) {
    double min = 0, max = 0;
    min = tab[0], max = tab[0];
    for (int i = 1; i < size; i++) {
        if (tab[i] < min) min = tab[i];
        else if (tab[i] > max) max = tab[i];
        else {}
    }
    printf("\n%0.10lf %0.10lf", min, max);
}

void agreguj(Grid* g) {
    for (int i = 0; i < g->nE; i++) {
        agregacja(&(g->element[i]));
    }
}

void obliczT(Grid* g){
    double** tab = new double*[g->nN];
    for (int z = 0; z < g->nN; z++) {
        tab[z] = new double[g->nN + 1] {0.0};
    }

    for (int z = 0; z < g->nN; z++) {
        for (int i = 0; i < g->nN; i++) {
            tab[z][i] = values[z][i];
        }
    }
    for (int i = 0; i < g->nN; i++) {
        tab[i][g->nN] = Pg[i];
    }
    for (int z = 0; z < g->nN; z++) {
        for (int i = z + 1; i < g->nN; i++) {
            for (int j = g->nN; j >= 0; j--) {
                tab[i][j] -= tab[z][j] * (tab[i][z] / tab[z][z]);
            }
        }
    }
}

```

```

    }
}

for (int i = g->nN - 1; i >= 0; i--) {
    t[i] = tab[i][g->nN];
    for (int j = i + 1; j < g->nN; j++) {
        t[i] -= tab[i][j] * t[j];
    }
    t[i] /= tab[i][i];
}

for (int i = 0; i < g->nN; i++) {
    delete[] tab[i];
}
delete[] tab;
}

void agregacja(Element* e) {
    for (int k = 0; k < 4; k++) {
        for (int j = 0; j < 4; j++) {
            values[e->values[k] - 1][e->values[j] - 1] += e->H[k][j] + e-
>Hbc[k][j];
            Cg[e->values[k] - 1][e->values[j] - 1] += e->C[k][j];
        }
        Pg[e->values[k] - 1] += e->p[k];
    }
}
};

```

Fragment kodu 10: struktura HGlobal

Struktura *HGlobal* zawiera wszystkie potrzebne do symulacji globalne dane: macierze $[H]$ oraz $[C]$, a także wektory $\{P\}$ oraz $\{T\}$. Oprócz konstruktora, tworzącego wszystkie atrybuty o właściwych rozmiarach, oraz destruktor, posiada potrzebne do symulacji funkcje oraz funkcję reprezentującą symulację.

- **findMaxMin()** - znajduje minimalną oraz maksymalną wartość temperatury elementu w całej siatce w danej iteracji symulacji
- **obliczT()** - rozwiązuje równanie (11) $[H]\{t\} + \{P\} = 0$ dla każdej iteracji symulacji wykorzystując metodę rozwiązywania układów równań liniowych - metodę eliminacji Gaussa (obliczanie wektora temperatur)
- **agreguj()**, **agregacja()** - funkcje tworzące wspólnie macierze globalne $[H]$ oraz $[C]$ wraz z wektorem globalnym $\{P\}$
- **symulacja()** - odpowiada za wykonywanie symulacji ustalonego procesu cieplnego, wykorzystuje wszystkie wyżej wymienione funkcje. W każdej iteracji obliczana jest macierz $[H] = [H] + [C]/dT$ oraz wektor $\{P\} = \{P\} + \{[C]/dT\} * \{T_0\}$, po czym obliczany jest wektor $\{T\}$ i znajduwane w nim minimum oraz maksimum, kolejno ta macierz i ten wektor są wypełniane zerami, a obliczony wektor $\{T\}$ staje się wektorem $\{T_0\}$ w kolejnym kroku czasowym

4. Wyniki testów

Przeprowadzono łącznie dziewięć testów - dla każdego z trzech plików testowych wykonano symulacje dla czterech, dziewięciu oraz szesnastu punktów całkowania.

4.1 Test1_4_4.txt

- **npc = 4**

110.03797235555065	365.81547262515920
168.83700976624803	502.59171427864766
242.80084627221015	587.37266670966756
318.61458870451042	649.38748218052217
391.25579178949226	700.06841829446569
459.03690891911020	744.06334147350560
521.58628539565746	783.38284621760943
579.03446139235552	818.99218357204552
631.68925823296991	851.43103779637056
679.90761913038693	881.05762938859448

Rysunek 4: wynik dla npc=4

- **npc = 9**

110.03797235555069	365.81547262515920
168.83700976624803	502.59171427864754
242.80084627221018	587.37266670966744
318.61458870451042	649.38748218052194
391.25579178949232	700.06841829446546
459.03690891911032	744.06334147350549
521.58628539565746	783.38284621760943
579.03446139235552	818.99218357204529
631.68925823296979	851.43103779637045
679.90761913038682	881.05762938859448

Rysunek 5: wynik dla npc=9

- **npc = 16**

105.83070087051398	365.34773918601752
151.55190295863835	492.94120507811505
209.59339233796072	568.57928926430009
269.43064489546026	622.33365874242884
327.06369928535713	665.44944160508680
381.08202738353344	702.40940015265892
431.11337012047568	735.15050522241756
477.19787274084945	764.61108612802093
519.53310217993214	791.31900405113504
558.36974144986618	815.62190143540556

Rysunek 6: wynik dla npc=16

4.2 Test2_4_4_MixGrid.txt

- npc = 4

95.01668045531817	377.02761672150223
148.14379066585667	516.92495320411069
220.68538020783353	601.75275099389989
296.89040334759216	662.79045736029104
370.64700656976731	712.25639951445851
439.79252432905366	755.07064641584509
503.75096510007722	793.31986350290208
562.56684294822946	827.97709549560500
616.51374949433296	859.56995161679379
665.93719624658934	888.44148729900712

Rysunek 7: wynik dla npc=4

- npc = 9

95.02234281620413	377.04392497713258
148.15446251721599	516.95301160900362
220.69886962869717	601.78625241922794
296.90498589914819	662.82531863918427
370.66166299091066	712.29050037736988
439.80673115474139	755.10302540395219
503.76448264898562	793.35015359278134
562.57957963361855	828.00521470069282
616.52568720356464	859.59594880391967
665.94835199596616	888.46546830213140

Rysunek 8: wynik dla npc=9

- npc = 16

92.07916425533681	383.34621046576683
133.65497595681066	516.28361668025877
191.15059125970853	592.68018266264255
251.78796037346729	645.69256605107080
310.68230456038248	687.68890619657327
366.06883640767938	723.52955594699552
417.43542168896465	755.25865513934843
464.77094410566400	783.83014833438983
508.25847243307692	809.76049133452523
548.14911421763429	833.38075369052581

Rysunek 9: wynik dla npc=16

4.3 Test3_31_31_kwadrat.txt

- npc = 4

100.00000000027184	149.55695180811625
100.00000000529101	177.44492795006863
100.00000005146053	197.26696292169996
100.00000033441943	213.15278729153133
100.00000163824052	226.68258341907574
100.00000647120355	238.60706480588087
100.00002152936499	249.34669194249932
100.00006221274084	259.16507915513046
100.00015978338752	268.24068900501453
100.00037134491939	276.70109786331943
100.00079223550657	284.64128318866727
100.00156984610132	292.13421905089575
100.00291748383819	299.23740994530652
100.00512679752832	305.99712152752318
100.00857746362753	312.45123021353044
100.01374321423224	318.63120613643798
100.02119375970858	324.56353148994350
100.03159261406788	330.27073917337373
100.04569120097801	335.77218904797962
100.06431986990380	341.08465853432125

Rysunek 10: wynik dla npc=4

- npc = 9

100.00000000027183	149.55695180811628
100.00000000529100	177.44492795006866
100.00000005146053	197.26696292169996
100.00000033441943	213.15278729153135
100.00000163824056	226.68258341907580
100.00000647120359	238.60706480588084
100.00002152936503	249.34669194249929
100.00006221274089	259.16507915513051
100.00015978338755	268.24068900501447
100.00037134491943	276.70109786331938
100.00079223550655	284.64128318866722
100.00156984610136	292.13421905089569
100.00291748383823	299.23740994530641
100.00512679752839	305.99712152752318
100.00857746362756	312.45123021353038
100.01374321423228	318.63120613643787
100.02119375970861	324.56353148994344
100.03159261406788	330.27073917337367
100.04569120097806	335.77218904797951
100.06431986990384	341.08465853432131

Rysunek 11: wynik dla npc=9

- npc = 16

```

100.00000000022020 147.78260833829609
100.00000000428707 172.79562494133739
100.000000004169644 189.99561450541162
100.00000027096699 203.56993056764708
100.00000132740233 215.03447195661963
100.00000524336380 225.08472624401281
100.00001744440466 234.10194115021443
100.00005040855714 242.32183288878451
100.00012946624626 249.90241924722443
100.00030088630328 256.95582994913980
100.00064191752860 263.56505656452134
100.00127198506168 269.79355987047620
100.00236392335333 275.69114747834124
100.00415404405886 281.29775511578259
100.00694998420076 286.64597971499427
100.01113559041794 291.76283395558198
100.01717247679778 296.67099618117061
100.02559826286218 301.38972246517500
100.03702179789892 305.93552611867102
100.05211588169020 310.32269321864624

```

Rysunek 12: wynik dla npc=16

5. Opracowanie wyników

Step	Ext	UPEL	npc = 4	npc = 9	npc = 16
50	min	110.03797659406167	110.03797235555065	110.03797235555069	105.83070087051398
	max	365.8154705784631	365.81547262515920	365.81547262515920	365.34773918601752
100	min	168.83701715655656	168.83700976624803	168.83700976624803	151.55190295863835
	max	502.5917120896439	502.59171427864766	502.59171427864754	492.94120507811505
150	min	242.80085524391868	242.80084627221015	242.80084627221018	209.59339233796072
	max	587.372666691486	587.37266670966756	587.37266670966744	568.57928926430009
200	min	318.61459376004086	318.61458870451042	318.61458870451042	269.43064489546026
	max	649.3874834542602	649.38748218052217	649.38748218052194	622.33365874242884
250	min	391.2557916738893	391.25579178949226	391.25579178949232	327.06369928535713
	max	700.0684204214381	700.06841829446569	700.06841829446546	665.44944160508680
300	min	459.03690325635404	459.03690891911020	459.03690891911032	381.08202738353344
	max	744.0633443187048	744.06334147350560	744.06334147350549	702.40940015265892
350	min	521.5862742337766	521.58628539565746	521.58628539565746	431.11337012047568
	max	783.382849723737	783.38284621760943	783.38284621760943	735.15050522241756
400	min	579.0344449687701	579.03446139235552	579.03446139235552	477.19787274084945
	max	818.9921876836681	818.99218357204552	818.99218357204529	764.61108612802093
450	min	631.6892368621455	631.68925823296991	631.68925823296979	519.53310217993214
	max	851.4310425916341	851.43103779637056	851.43103779637045	791.31900405113504
500	min	679.9075931513394	679.90761913038693	679.90761913038682	558.36974144986618
	max	881.057634906017	881.05762938859448	881.05762938859448	815.62190143540556

Tabela 2: Zestawienie wyników testu dla pliku Test1_4_4.txt

Dla wszystkich kroków czasowych dla **npc=4** a **npc=9** wszystkie wyniki są niemalże identyczne (identyczne z dokładnością 10^{-12}) zestawiając je z wynikami UPEL. Natomiast **npc=16** początkowo wykazuje poprawne wartości maksymalne, które z czasem coraz bardziej odbiegają od wyników UPEL, tak samo jak wartości minimalne. Wartość minimalna w pierwszej iteracji dla **npc=16** różni się o ok. 5°C .

Step	Ext	UPEL	npc = 4	npc = 9	npc = 16
50	min	95.15184673458245	95.01668045531817	95.02234281620413	92.07916425533681
	max	374.6863325385064	377.02761672150223	377.04392497713258	383.34621046576683
100	min	147.64441665454345	148.14379066585667	148.15446251721599	133.65497595681066
	max	505.96811082245307v	516.92495320411069	516.95301160900362	516.28361668025877
150	min	220.1644549730314	220.68538020783353	220.69886962869717	191.15059125970853
	max	586.9978503916302	601.75275099389989	601.78625241922794	592.68018266264255
200	min	296.7364399006366	296.89040334759216	296.90498589914819	251.78796037346729
	max	647.28558387732	662.79045736029104	662.82531863918427	645.69256605107080
250	min	370.968275802604	370.64700656976731	370.66166299091066	310.68230456038248
	max	697.3339863103786	712.25639951445851	712.29050037736988	687.68890619657327
300	min	440.5601440058566	439.79252432905366	439.80673115474139	366.06883640767938
	max	741.2191121514377	755.07064641584509	755.10302540395219	723.52955594699552
350	min	504.8911996551285	503.75096510007722	503.76448264898562	417.43542168896465
	max	781.209569726045	793.31986350290208	793.35015359278134	755.25865513934843
400	min	564.0015111915015	562.56684294822946	562.57957963361855	464.77094410566400
	max	817.3915065469778	827.97709549560500	828.00521470069282	783.83014833438983
450	min	618.1738556427995	616.51374949433296	616.52568720356464	508.25847243307692
	max	850.2373194670416	859.56995161679379	859.59594880391967	809.76049133452523
500	min	667.7655470268747	665.93719624658934	665.94835199596616	548.14911421763429
	max	880.1676054000437	888.44148729900712	888.46546830213140	833.38075369052581

Tabela 3: Zestawienie wyników testu dla pliku Test2_4_4_MixGrid.txt

Dla wszystkich kroków czasowych dla **npc=4** wyniki minimalne są stosunkowo dokładne (różnica maksymalnie 2°C), gdy zestawione zostaną z wynikami UPEL. Podobnie sytuacja wygląda z wartościami maksymalnymi, gdzie różnica wyników wraz z każdym krokiem się zwiększa (największa różnica wynosi aż 8°C). Wartości obliczone z wykorzystaniem **npc=9** są w tym teście również bardzo bliskie **npc=4**. Natomiast **npc=16** już w pierwszych iteracjach wykazuje błędy kilku stopni w wartościach maksymalnych oraz minimalnych, których margines błędu zwiększa się wraz z każdym krokiem czasu.

Step	Ext	UPEL	npc = 4	npc = 9	npc = 16
1	min	99.99969812978378	100.00000000027184	100.00000000027183	100.00000000022020
	max	149.5566275788947	149.55695180811625	149.55695180811628	147.78260833829609
2	min	100.00053467957446	100.00000000529101	100.00000000529100	100.00000000428707
	max	177.44482649738018	177.44492795006863	177.44492795006866	172.79562494133739
3	min	100.00084733335379	100.00000005146053	100.00000005146053	100.00000004169644
	max	197.2672291500534	197.26696292169996	197.26696292169996	189.99561450541162
4	min	100.00116712763896	100.00000033441943	100.00000033441943	100.00000027096699
	max	213.15348263983788	213.15278729153133	213.15278729153135	203.56993056764708
5	min	100.00150209858216	100.00000163824052	100.00000163824056	100.00000132740233
	max	226.6837398631218	226.68258341907574	226.68258341907580	215.03447195661963
6	min	100.001852708951	100.00000647120355	100.00000647120359	100.00000524336380
	max	238.60869878203812	238.60706480588087	238.60706480588084	225.08472624401281
7	min	100.00222410506852	100.00002152936499	100.00002152936503	100.00001744440466
	max	249.34880985057373	249.34669194249932	249.34669194249929	234.10194115021443
8	min	100.00263047992797	100.00006221274084	100.00006221274089	100.00005040855714
	max	259.1676797521773	259.16507915513046	259.16507915513051	242.32183288878451
9	min	100.00310216686808	100.00015978338752	100.00015978338755	100.00012946624626
	max	268.24376548847937	268.24068900501453	268.24068900501447	249.90241924722443
10	min	100.00369558647527	100.00037134491939	100.00037134491943	100.00030088630328
	max	276.70463950306436	276.70109786331943	276.70109786331938	256.95582994913980
11	min	100.00450560745507	100.00079223550657	100.00079223550655	100.00064191752860
	max	284.64527660833346	284.64128318866727	284.64128318866722	263.56505656452134
12	min	100.00567932588369	100.00156984610132	100.00156984610136	100.00127198506168
	max	292.1386492100023	292.13421905089575	292.13421905089569	269.79355987047620
13	min	100.00742988613344	100.00291748383819	100.00291748383823	100.00236392335333
	max	299.242260871447	299.23740994530652	299.23740994530641	275.69114747834124
14	min	100.01004886564658	100.00512679752832	100.00512679752839	100.00415404405886
	max	306.00237684844643	305.99712152752318	305.99712152752318	281.29775511578259
15	min	100.01391592562979	100.00857746362753	100.00857746362756	100.00694998420076
	max	312.4568735346492	312.45123021353044	312.45123021353038	286.64597971499427
16	min	100.01950481085419	100.01374321423224	100.01374321423228	100.01113559041794
	max	318.637221302136	318.63120613643798	318.63120613643787	291.76283395558198
17	min	100.02738525124852	100.02119375970858	100.02119375970861	100.01717247679778
	max	324.56990275925733	324.56353148994350	324.56353148994344	296.67099618117061
18	min	100.0382207726261	100.03159261406788	100.03159261406788	100.02559826286218
	max	330.27745133351596	330.27073917337373	330.27073917337367	301.38972246517500
19	min	100.05276279329537	100.04569120097801	100.04569120097806	100.03702179789892
	max	335.77922748329735	335.77218904797962	335.77218904797951	305.93552611867102
20	min	100.07184163487159	100.06431986990380	100.06431986990384	100.05211588169020
	max	341.0920092636545	341.08465853432125	341.08465853432131	310.32269321864624

Tabela 4: Zestawienie wyników testu dla pliku Test3_31_31_kwadrat.txt

Wszystkie wartości dla **npc=4** oraz **npc=9** są niemalże identyczne, co podkreśla skuteczność tych metod w tym teście. Natomiast dla **npc=16** występuje takie samo zachowanie jak w poprzednich testach - wartości maksymalne już w pierwszej iteracji mają margines błędu, który się zwiększa coraz bardziej. Trzeba przyznać, że wyniki minimalne są poprawne (różnica w mało znaczących liczbach), jednakże są one wciąż mniej dokładne niż dla **npc=4** i **npc=9**.

6. Analiza wyników

Przeprowadzona analiza symulacji ustalonych procesów cieplnych przy użyciu Metody Elementów Skończonych (MES) pozwoliła na ocenę dokładności i efektywności różnych schematów całkowania ($npc = 4, 9, 16$) oraz wpływu parametrów siatki na uzyskane wyniki.

Zbieżność wyników z UPEL:

- Dla **npc=4** i **npc=9**, wyniki były bardzo zbliżone do wyników referencyjnych, co wskazuje na wystarczającą dokładność tych schematów całkowania.
- **npc=16** wykazuje większe odchylenia, szczególnie w wartościach maksymalnych dla późniejszych kroków czasowych.

Wpływ liczby punktów całkowania:

- **npc=4**: Schemat cechuje się dobrą dokładnością dla minimalnych wartości temperatury, ale odchylenia w wartościach maksymalnych są zauważalne, zwłaszcza przy rzadszych siatkach.
- **npc=9**: Oferuje najlepszy kompromis między dokładnością a kosztem obliczeniowym, wyniki są stabilne i zgodne z referencją.
- **npc=16**: Paradoksalnie, przy wyższej liczbie punktów całkowania wyniki stają się mniej zgodne z UPEL, co może wynikać z akumulacji błędów numerycznych.

Wpływ gęstości siatki:

- Gęstsza siatka (31x31_kwadrat) pozwala na stabilniejsze wyniki i mniejsze różnice między schematami całkowania. Dla rzadszych siatek (4x4 i 4x4_MixGrid) różnice między **npc=4**, **npc=9**, a **npc=16** były bardziej widoczne.

Stabilność i dokładność:

- Wartości minimalne są generalnie bardziej stabilne i mniej podatne na różnice w schematach całkowania, natomiast wartości maksymalne wykazują większe odchylenia wraz ze wzrostem kroku czasowego.

7. Wnioski

Optymalny schemat całkowania:

- **npc=9** wydaje się najbardziej odpowiednim wyborem, zapewniającym wysoką zgodność z wynikami referencyjnymi przy akceptowalnym koszcie obliczeniowym. Nie ma wyraźnych korzyści z zastosowania **npc=16** w analizowanych przypadkach.

Wpływ parametrów siatki:

- Przy rzadszych siatkach konieczne jest stosowanie bardziej precyzyjnych schematów całkowania (np. **npc=9**), aby zminimalizować błędy wynikające z dyskretyzacji. Gęstsze siatki mogą poprawić stabilność wyników nawet przy prostszych schematach.

Ograniczenia MES:

- Zwiększenie liczby punktów całkowania (np. **npc=16**) może prowadzić do większej akumulacji błędów numerycznych, szczególnie w przypadku skomplikowanych warunków brzegowych lub mniej precyzyjnych operacji macierzowych.

Praktyczne zastosowanie:

- MES w symulacji ustalonych procesów cieplnych jest narzędziem skutecznym i elastycznym, ale wymaga uwzględnienia kompromisu między dokładnością a czasem obliczeń, szczególnie przy wyborze schematów całkowania i gęstości siatki.

Rekomendacje:

- W przyszłych analizach warto dokładniej zbadać wpływ różnych typów warunków brzegowych na wyniki symulacji.
- Zaleca się także uwzględnienie bardziej zaawansowanych metod redukcji błędów numerycznych, szczególnie przy wyższych **npc**.